PLATFORM FOR AUTONOMOUS INDOOR NAVIGATION USING SLAM, META-RL, AND MULTI-ROBOT COLLABORATION

25-26J-497

Project Proposal Report

GOMIS R J S – IT22349606

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

August 2025

PLATFORM FOR AUTONOMOUS INDOOR NAVIGATION USING SLAM, META-RL, AND MULTI-ROBOT COLLABORATION

25-26J-497

Project Proposal Report

GOMIS R J S – IT22349606

Supervisor: Mr. Samadhi Rathnayake

Co-Supervisor: Mr. Amila Alexander

BSc (Hons) Degree in Information Technology

Specialising in Software Engineering

Department of Software Engineering

August 2025

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
GOMIS R J S	IT22349606	Gomis

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

28/8/25

Signature of the Supervisor

Date

Signature of the Co-Supervisor

Date

ABSTRACT

The advancement of autonomous navigation systems, particularly within complex indoor environments, is critically dependent on a robot's ability to perceive and identify objects in its surroundings. This perception is foundational for sophisticated tasks such as semantic SLAM and adaptive navigation policies driven by Meta-Reinforcement Learning (Meta-RL). However, training the deep learning models required for robust object identification is often hindered by the immense cost and labor associated with collecting and manually annotating large-scale, real-world datasets. This research component addresses this critical bottleneck by developing a systematic framework for generating high-quality synthetic datasets from virtual environments to train an efficient, real-time object detection model. The project will utilize the Gazebo simulation platform integrated with ROS 2 to create diverse indoor scenarios. A key focus will be the implementation of domain randomization techniques to bridge the "sim-to-real" gap, ensuring the trained model can generalize effectively to real-world conditions. A state-of-the-art YOLOv8 model will be trained on this synthetic data and deployed as a ROS 2 node, providing a modular and high-performance perception system. The final output will be a well-validated object detection model and a reusable data generation pipeline that serves as a critical perception input for the broader autonomous navigation platform.

Keywords – Gazebo Platform, YOLO-V8, Robot Operating System, Meta-RL, SLAM, Autonomous Navigation

TABLE OF CONTENT

Declaration i

Abstract ii

TABLE OF CONTENT ii

LIST OF TABLES v

1 Introduction 1

1.1 Background and Literature Survey 2

1.1.1 Challenge 1: Real-Time Adaptation in Dynamic Environments 3

1.1.2 Challenge 2: Decentralized Coordination and Collaboration 3

1.1.3 Challenge 3: System Complexity and Operational Oversight 4

1.2 Research Gap 6

1.2.1 Lack of Integrated Workflows 6

1.2.2 Sim-to-Real Generalization 6

1.2.3 Performance on Low-Fidelity Objects 6

1.2.4 Integration Gap and Need for a Comprehensive Platform 6

1.3 Research Problem 9

1.3.1 The Data Acquisition Bottleneck 9

1.3.2 The Reality Gap 10

1.3.3 The Integration Challenge 10

1.3.4 The Performance Mandate 11

2 Research Objectives 11

2.1 Main Objectives 11

2.2 Specific Objectives – Multi-Robot Collaboration Framework 13

2.2.1 Develop a Configurable Virtual Environment for Data Generation .. 13

2.2.2 Implement an Automated Synthetic Data Generation Pipeline 13

2.2.3 Train and Optimize a State-of-the-Art Object Detection Model..... 13
2.2.4 Deploy and Validate the Trained Model in a ROS 2 Environment 13
3 Methodology 14
3.1 System Overview 14
3.2 Software Solutions 16
3.2.1 Requirement Gathering and Analysis 16
3.2.2 Feasibility Study 18
3.3 Tasks and Sub-Tasks 19
3.4 Testing and Validation 19
3.5 Data Collection..... 20
3.6 Time Frame & Schedule 21
4 Project Requirements 22
4.1.1 Functional Requirements 22
4.1.2 Non-Functional Requirements 22
4.1.3 System Requirements..... 22
References 24

LIST OF FIGURES

Figure 3-1 System Architecture Diagram 14
Figure 3-2 Component Diagram 16
Figure 3-3: Gantt Chart 21

LIST OF TABLES

Table 1.1 Core Challenges in Multi-Robot System Deployment..... 5
Table 1.2 Research Gap Analysis..... 7

LIST OF ABBREVIATIONS

Abbreviations	**Description**
ROS	Robot Operation System
SLAM	Simultaneous Localization and Mapping
Meta-RL	Meta-reinforcement learning
AGV	Automated Guided Vehicles
LiDAR	Light Detection and Ranging
DRL	Deep Reinforcement Learning
URDF	Unified Robot Description Format
IMU	Inertial Measurement Unit

1 INTRODUCTION

Autonomous indoor navigation in unknown and dynamic environments has long been a challenge in robotics, but it is becoming increasingly important in areas such as smart homes, automated warehouses and disaster recovery scenarios. The ability of robots to travel successfully without prior maps is significantly relied on SLAM, a technique that allows for real time environment reconstruction while tracking the robot's position [1]. SLAM has made significant advances with optical, LiDAR and multi sensor fusion techniques providing robust performance in controlled environments. However, most traditional SLAM algorithms assume static or semi static surroundings and a single robot operational model, limiting their effectiveness in real world applications where settings are fundamentally dynamic [4].

Dynamic components including moving individuals, moving furniture, clear surfaces and reflective materials pose significant problems to typical SLAM systems [3]. These issues appear as false loop closures, map inconsistencies, and loss of localization accuracy which could risk navigation safety and efficiency. Rapid adaptation is crucial in highly dynamic environments, yet standard rule-based path planning and control systems frequently fail to respond successfully [5].

Meta-RL has emerged as a potential method for rapid robotic adaptability. Unlike the traditional reinforcement learning methods, Meta-RL allows systems to make assumptions from previous experiences and quickly modify navigation strategies to the new unknown environments [9][10]. Meta-RL has demonstrated great potential for single robot navigation but how it can be used in a multi-robot collaborative context is mostly unexplored due to various reasons. This gap is significant since collaborative navigation offers various advantages such as simultaneous exploration for efficient mapping, shared situational awareness and resilience through distributed decision-making [6]-[8].

The multi-robot collaboration framework is a key component for these advantages. It allows real time map sharing, task allocation, conflict resolution and synchronized exploration. By merging local maps into a consistent global map [6], robots can leverage one another's discoveries. Our framework operationalizes these requirements

with decentralized coordination and robust map fusion and scheduling mechanisms, designed for heterogeneous robot teams [7].

This proposal introduces an integrated platform for Autonomous Indoor Navigation by combining advanced SLAM with Meta-RL based adaptation engine, a decentralized multi robot collaborative framework and a 3D mapping and visualization platform. The system utilizes an edge-cloud architecture, in which lightweight robots with simple sensors transporting heavy computations like SLAM processing, map fusion, and policy optimization to a centralized server. This design offers scalability, cost-efficiency, and interoperability with a variety of robot types while preserving real-time performance [6].

From a software engineering approach, the platform encourages flexibility, reusability, and scalability to facilitate the integration of emerging algorithms, sensors, and robotic platforms. Along with SLAM, Meta-RL, and the multi-robot cooperation architecture, the 3D Mapping & Visualization Platform offers an easy-to-use interface for real-time monitoring and debugging for operators. These components improve autonomous robotics by allowing multi-robot teams to move collaboratively, adapt to dynamic changes, and keep consistent global maps in complicated interior environments. The suggested solution bridges the gap between academic research and industry implementation, increasing logistical efficiency, ensuring safety in hazardous areas, and allowing new robotics applications in human-centric environments.

1.1 Background and Literature Survey

The evolution of autonomous systems has altered the way indoor logistics, inspection and service activities are carried out. Historically, tasks like item transportation and inventory management relied significantly on human labor which was inefficient, physically demanding and error prone. Automated Guided Vehicles (AGVs) were the first wave of automation to enable repeated activities in organized and controlled environments. However, AGVs depended on permanent navigation infrastructure, such as magnetic strips or painted floor lines, rendering them inefficient in the places with layout changes and dynamic obstacles [1].

The introduction of Autonomous Mobile Robots (AMRs) marked a significant leap forward. AMRs can dynamically detect and traverse new surroundings by utilizing advanced sensing methods such as LiDAR, RGB-D cameras and inertial measurement units (IMUs) as well as SLAM algorithms [2][3]. This enables its usage in complicated, real-world environments such as warehouses, hospitals and smart buildings. However, in large-scale applications a single AMR is insufficient. The true potential lies in deploying fleets of robots that can work together to explore and operate more effectively. This transition from a single agent to multi agent system creates a new set of challenges that define the boundary of current robotics research [6],[7].

1.1.1 Challenge 1: Real-Time Adaptation in Dynamic Environments

Indoor environments are hardly static because moving people, rearranging furniture and temporary obstacles cause a shift in distribution between training and operating settings. While Deep Reinforcement Learning (DRL) has shown ordinary DRL agents require considerable training in new settings, making them not suitable for applications that require immediate attention. Meta-RL solves this problem by allowing agents to "learn how to learn" and change rules with little new knowledge [9][10]. For example, Google research conducted in 2020 demonstrated fast adaptability in legged robots through evolutionary meta-learning [11]. However, developing whole lifecycle platforms from simulation-based training to real world deployments and constant fine-tuning is still an unexplored topic.

1.1.2 Challenge 2: Decentralized Coordination and Collaboration

Scaling from a single robot to a collaborative team raises serious distributed system challenges. To outperform a single agent robot, a team of robots must coordinate their activities to achieve a shared goal, such as exploring an unexplored region. This requires addressing several key challenges:

- **Communication and Data Fusion**: Agents must effectively share sensor data and local maps with each other. Combining these incomplete maps into a single global map is a difficult data fusion task [6]. Systems such as CORB-SLAM have created frameworks for this but they frequently rely on protocols that may struggle with scalability and network instability.

- **Dynamic Task Allocation**: Dynamic task allocation is appointing different agents for different areas of a building to explore in order to increase coverage and reduce redundancy. Cooperative navigation techniques are vital for efficiency but building decentralized algorithms that lead to globally optimum behavior remains a persistent difficulty [8].
- **Conflict Resolution**: Managing path conflicts and right-of-way to prevent collisions and deadlocks in shared environments.

These challenges require robust communication protocols, consensus algorithms, and lightweight yet scalable data-sharing strategies.

1.1.3 Challenge 3: System Complexity and Operational Oversight

A multi-robot fleet generates a continuous stream of high-dimensional real-time data which includes raw sensor feeds, SLAM-generated maps, navigation choices, job allocations and inter-robot communication. Without the appropriate tools, this data can become overwhelming for operators, which can make the system impossible to analyze, diagnose and optimize. Existing interfaces often provide only basic 2D positional data, lacking advanced features such as collision heatmaps, historical performance analytics, and interactive map annotations [12].

The absence of integrated visualization and diagnostic tools in many existing multi-robot frameworks is a major obstacle to scaling. Without them, troubleshooting involves manual log analysis, which is time-consuming and prone to error.

To address this difficulty, a 3D Mapping & Visualization Platform must be developed that not only depicts real-time global maps and robot data, but also includes historical analytics, performance heatmaps, and interactive controls. Such a system converts raw sensor and navigation data into usable insights, allowing operators to make quick decisions, save downtime, and retain confidence in the system's performance even if the number of robots and mission complexity increases.

Table 1.1 Core Challenges in Multi-Robot System Deployment

Factor	Practical Effect on Robots	Typical Symptom	Impact (1-5)
Real-Time Adaptation and Dynamic Obstacles	Inability to adjust to new or moving obstacles	Collisions, inefficient pathing, operational freezes	5
Multi-Robot Coordination and Scalability	Robots performing redundant work or interfering with each other	Inefficient area coverage, inconsistent global maps, bottlenecks	5
System Complexity and Operational Oversight	Inability for human operators to monitor, debug, or manage the fleet	Prolonged downtime, difficulty diagnosing failures, lack of trust in the system	4

The available literature typically addresses these issues in isolation. Meta-RL research may produce a highly adaptable agent, but it fails to investigate how it operates in multi-agent systems. Working on collaborative SLAM may help with map merging, but it assumes robots with basic, non-adaptive behaviors. The key gap therefore is the lack of a single, integrated platform to address these concerns comprehensively. Without a strong framework to regulate these interactions, a team of robots may be less efficient than a single agent. This difficulty is essentially a distributed systems engineering problem, requiring solutions that assure data consistency and allow for dynamic resource allocation across independent agents.

1.2 Research Gap

Even though the ideas of object detection and synthetic data generation are well-established, there are still a number of gaps, especially when it comes to developing effective and modular perception systems for adaptive autonomous navigation.

1.2.1 Lack of Integrated Workflows

The architecture of a new object detection model or the complexities of a new simulation technique are the main topics of most of the academic research. Comprehensive end-to-end protocols that lead a researcher from virtual environment setup to the deployment of a validated, real-time ROS 2 node are lacking. Projects find it challenging to transition from theory to practice because of this integration gap.

1.2.2 Sim-to-Real Generalization

While domain randomization is a well-known approach, its successful implementation remains a topic of ongoing study. The appropriate values and forms of randomization required for certain interior situations are frequently determined by trial and error. There is a need for a more systematic strategy to implementing and assessing these strategies' effects on model performance.

1.2.3 Performance on Low-Fidelity Objects

Many synthetic datasets emphasize high-quality, lifelike 3D elements. However, in many practical contexts (including quick prototyping for research), settings are created using simpler, lower-fidelity items. Research is needed to determine how successfully models trained on simplified virtual assets can learn the properties required for real-world item detection.

1.2.4 Integration Gap and Need for a Comprehensive Platform

While individual components such as accurate SLAM, adaptive navigation, collaborative mapping, and change detection are mature in the literature, they are frequently used in isolation. There is no single, integrated platform that integrates:

- Real-time Meta-RL for adaptive navigation.
- Decentralized multi-robot collaboration with dynamic task allocation and conflict resolution.

- 3D SLAM mapping.
- Executes real-time, version-controlled change detection across a fleet.

To address these gaps a modular, edge-cloud platform that can integrate capabilities is required, allowing robot teams to change quickly, collaborate efficiently and maintain accurate shared maps while staying operationally scalable and robust.

Table 1.2 Research Gap Analysis

Research Study	**Focus Area**	**Strengths**	**Gap**	**Relation to Proposed Comprehensive Platform**	**Reference**
ORB-SLAM	3D SLAM	Accurate localization, loop closure.	Static environments, single robots, lacks dynamic adaptation.	Forms SLAM backbone and extend for multi-robot.	[13]
RTAB-Map	Real-time RGB-D SLAM	Loop closure, real-time mapping.	Limited scalability, single-robot focus, centralized.	Used as backend for collaborative multi-robot SLAM.	[14]
Cartographer	LiDAR SLAM	Real-time 2D/3D mapping	High computation, minimal dynamic obstacle handling.	Integrated into hybrid platform with adaptive obstacle awareness	[15]
MAML	Meta-RL	Rapid generalization of unseen tasks	Simulation only; lacks real-world integration, single robot	Provides Meta-RL adaptive navigation engine	[16]
CORB-SLAM	Multi-robot distributed SLAM	Map fusion, distributed architecture	Network-dependent, centralized task allocation, no adaptive policies	Inspiration for decentralized multi-robot collaboration framework	[6], [18]

DDF-SLAM	Multi-robot SLAM	Data sharing and distributed mapping	limited dynamic adaptation	Adapted for heterogeneous robot teams with adaptive path planning	[19]

The gaps clearly demonstrate the need for a comprehensive platform that integrates these features.

1.3 Research Problem

This project's main research question is: How can an autonomous indoor robot's high-performance, real-time object detection model be effectively developed using primarily synthetic data, avoiding the prohibitive costs and logistical difficulties of extensive real-world data collection?

Even while autonomous robotics has advanced significantly, it is still difficult for robots to semantically comprehend their surroundings, that is, to recognize impediments rather than merely observe them. There is a substantial barrier to entry for research and development since the present paradigm for training robust perception models mostly depends on large, manually annotated datasets. By developing a comprehensive solution using a virtualized data creation and model deployment pipeline, this project seeks to remove that obstacle. This overarching problem can be deconstructed into four critical and interconnected sub-problems:

1.3.1 The Data Acquisition Bottleneck

Data is what all current computer vision systems are built on. However, there are fundamental flaws in the way this data is obtained and prepared for robotic applications. To gather raw photos, a robot must be manually navigated through a variety of surroundings for hours or days. This is followed by an even more time-consuming procedure in which human annotators must painstakingly draw bounding boxes around each object of interest in thousands of frames. This is not a one-time task; without a fresh dataset, a model designed for a warehouse won't function in an office. This results in a debilitating bottleneck:

- Economic inefficiency: For a single, moderately sized dataset, the cost of human annotation services might reach thousands of dollars, rendering it impractical for quick prototyping or academic study.
- Time inefficiency: The iterative cycle of research and development is significantly slowed down by the turnaround time for data collecting and labeling, which can range from weeks to months.
- Lack of Diversity and Edge Cases: Rare but crucial conditions (like as an object partially veiled in a certain way) or hazardous circumstances that are necessary

for training a properly robust model are frequently missed by real-world data collecting.

1.3.2 The Reality Gap

Generating data from a simulator is the most straightforward way to address the data acquisition bottleneck. Data is unbounded, unrestricted, and precisely labeled in a virtual environment. But there is a big difference between the chaotic, complex realities of the real world and the clean, predictable mechanics of a simulation. Subtle variations in texturing, lighting physics, sensor noise, and item diversity are examples of this "sim-to-real" gap. When used on a real robot, a model that was trained only on synthetic data frequently gets "over-fitted" to the peculiarities of the simulator and is unable to generalize, which results in a sharp decline in performance. Therefore, this project needs to deal with how to generate "real enough" synthetic data to generate a model that works well in the real world.

1.3.3 The Integration Challenge

In isolation, an object detection model is meaningless. Perception is a crucial decision-making input for a mobile robot, but it is not the ultimate objective. Consequently, the model needs to be designed as a smooth, real-time part of a bigger robotic ecosystem rather than as a stand-alone application. The Robot Operating System (ROS 2), the industry standard for robotics software development, will be used in this project. This presents a number of engineering difficulties:

- Real-Time Data Handling: A high-frequency stream of camera data must be efficiently subscribed to by the system.
- Inter-Process Communication: It must process this data and post the findings (locations, object classes) into a ROS 2 topic in a standardized, structured message format that other nodes, like the Meta-RL (navigation) agent and the SLAM (mapping) module, may use right away.
- Reusability and Modularity: The finished perception node need to be a self-contained, reusable module that integrates smoothly and quickly into the bigger team effort.

1.3.4 The Performance Mandate

When navigating a dynamic environment, an autonomous robot must be able to observe its surroundings and respond in a matter of milliseconds. In addition to being ineffective, a perception system that takes too long to analyze a picture might be harmful. A crucial lag occurs between the moment the camera "sees" an obstruction and the moment the navigation system is "told" about it. The object detection model is thus subject to stringent real-time limitations. It needs to be quick and computationally light, not just accurate. This difficulty entails:

- Algorithmic Efficiency: Selecting a model architecture specifically intended for high-speed inference, such as YOLOv8.
- Resource management is making sure the model can function well on the constrained computing hardware that is usually present on a mobile robot (such as an NVIDIA Jetson or even a potent CPU) without using up all of the resources required for other crucial procedures like control and motion planning.

By developing, putting into practice, and thoroughly validating a comprehensive pipeline for the creation of synthetic data and the deployment of models, this research component seeks to provide a comprehensive solution to these interconnected issues. The result will be a useful and efficient substitute for conventional data collection techniques, directly supporting the project's more complex SLAM and adaptive navigation objectives.

2 RESEARCH OBJECTIVES

2.1 Main Objectives

The main objective of this project is to create, build, and evaluate an integrated Autonomous Indoor Navigation Platform that can operate efficiently in dynamic and unfamiliar environments. The proposed platform will combine 3D SLAM-based mapping, Meta-RL for rapid obstacle-aware adaptation, a decentralized multi-robot collaboration architecture, and an interactive 3D mapping and visualization interface to establish a single, modular ecosystem.

The goal is to provide a platform that enables multiple teams of autonomous robots to collaborate as a cohesive unit, adjusting their navigation techniques in real time, keeping consistent global maps, and efficiently coordinating missions without repeating effort. The integration of SLAM and Meta-RL allows each robot to see and adapt to its surroundings independently, while the collaboration architecture ensures that the team maximizes coverage, avoids conflicts, and effortlessly communicates mapping data. The 3D Mapping & Visualization Platform will give operators a simple, real-time interface for tracking fleet performance, view collision maps, reviewing mission data, and interacting with the system as needed.

From a systems engineering standpoint, the study emphasizes scalability, flexibility and adaptability, guaranteeing that the platform can integrate new algorithms, sensors, and robot types with little reconfiguration. The framework will allow for both simulation-based training and real-world deployment, bridging the gap between academic research and industrial applications including warehouse automation, facility monitoring or smart house navigation. Finally, this system will serve as a model for future autonomous indoor navigation technologies, combining adaptability, collaboration, and operational oversight into a single, integrated solution.

2.2 Specific Objectives – Multi-Robot Collaboration Framework

2.2.1 Develop a Configurable Virtual Environment for Data Generation

- Using a target set of typical home or warehouse items, create one or more intricate indoor scenes in the Gazebo simulator.
- To build a model of a camera-equipped robot in URDF (Unified Robot Description Format) and include it into the virtual world.

2.2.2 Implement Automated Synthetic Data Generation Pipeline

- To create scripts that gather visual data from the robot's virtual camera automatically.
- To use procedurally changing textures, lighting, and item placements in the simulation to use domain randomization approaches.
- To guarantee that all target objects in the collected photos have pixel-perfect bounding box annotations automatically generated by the pipeline.

2.2.3 Train and Optimize a State-of-the-Art Object Detection Model

- To train a YOLOv8 object identification model using the artificial dataset that was created.
- To maximize the accuracy and generalization potential of the model by experimenting with various training hyperparameters and data augmentation techniques.

2.2.4 Deploy and Validate the Trained Model in a ROS 2 Environment

- To create a Python node for ROS 2 that loads the custom model that has been trained.
- In addition to doing real-time object identification and publishing the findings (class labels, confidence scores, and bounding boxes) to a specific ROS 2 topic, this node will subscribe to a simulated camera feed.
- To verify the model's performance both qualitatively through RViz2 visualization and statistically using common measures (mAP, IoU).

3 METHODOLOGY

This project uses an organized, iterative methodology that starts with data creation and environment setup, progresses through model training, and ends with deployment and validation. Before incorporating each component into the pipeline's subsequent stage, this guarantees that it is reliable. By setting measurable milestones, this phased approach aims to control complexity and reduce project risk. To guarantee that possible problems are recognized and fixed early in the development of a lifecycle rather than during final integration, each phase generates a tangible output that forms the basis for the subsequent one. From the standpoint of software engineering, this method promotes reusability, flexibility, and modularity, enabling ongoing development at every stage. A data-centric approach at the heart of this methodology turns a simulated environment into a high-performance perception system that can be integrated into a more extensive autonomous navigation platform.

3.1 System Overview

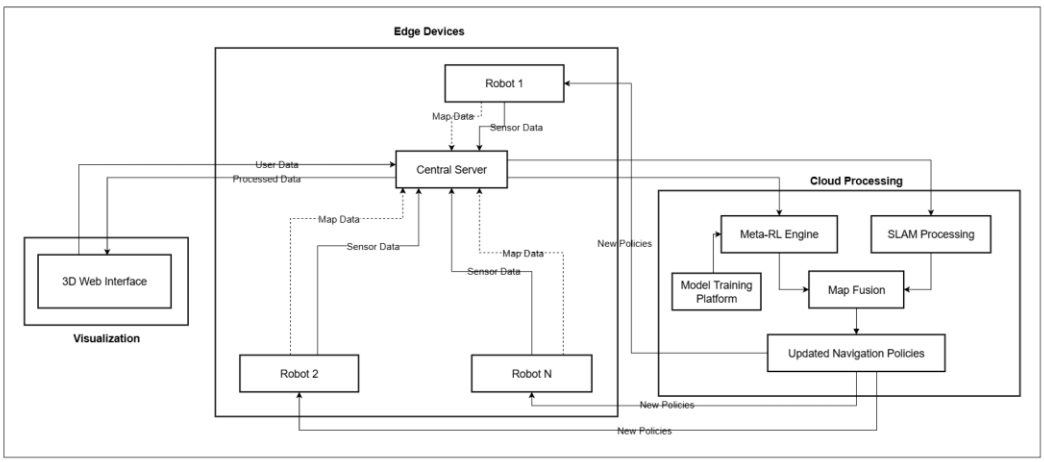

Figure 3-1 System Architecture Diagram

As shown in Figure 3.1, the architecture of this research component is best described as a data and model pipeline. A simulated world is converted into a deployable, real-time perception system through a modular and reproducible methodology. The pipeline consists of the following key stages:

Simulation Environment (Gazebo): This is the foundation of the pipeline. A 3D virtual world is created containing the indoor layout, furniture, and a set of target

objects to be identified. A simulated mobile robot, equipped with a virtual camera, is placed within this environment.

Data Generation & Augmentation Engine: A collection of Python scripts that manages the simulation and produces a wide range of data.

- Scenario Controller: To take pictures from multiple angles, this script programmatically moves the robot to different places and orientations.
- Domain Randomizer: This module modifies the simulation's attributes at each stage. It might alter the lighting's hue and intensity, switch up the wall and floor textures, or shift the things a little.
- Data Collector & Annotator: This script stores the camera's pictures and, most importantly, asks the Gazebo simulation engine for the exact 2D bounding boxes of the objects that are in the camera's field of vision. The YOLO annotation format is used to store this data.

Model Training (Ultralytics YOLOv8): The YOLOv8 training framework receives the curated, annotated synthetic dataset. This step is carried out offline on a computer that is capable (preferably with a GPU). The neural network's weights are modified by the framework using the dataset until the object locations can be predicted with accuracy. A single best.pt file with the weights of the trained mode is the output.

Real-Time Inference (ROS 2 Node): A ROS 2 node is used to deploy the finished, trained model.

- This node subscribes to the /camera/image_raw topic that the robot (either a real robot or a simulation) publishes.
- It uses the loaded model to do inference for every incoming image.
- The detection results, which include a list of items found, their bounding box coordinates, and their confidence levels, are subsequently published to a /detections topic. The perceptual data for the remaining autonomous navigation system components is provided by this topic, which functions as the project's output.

3.2 Software Solutions

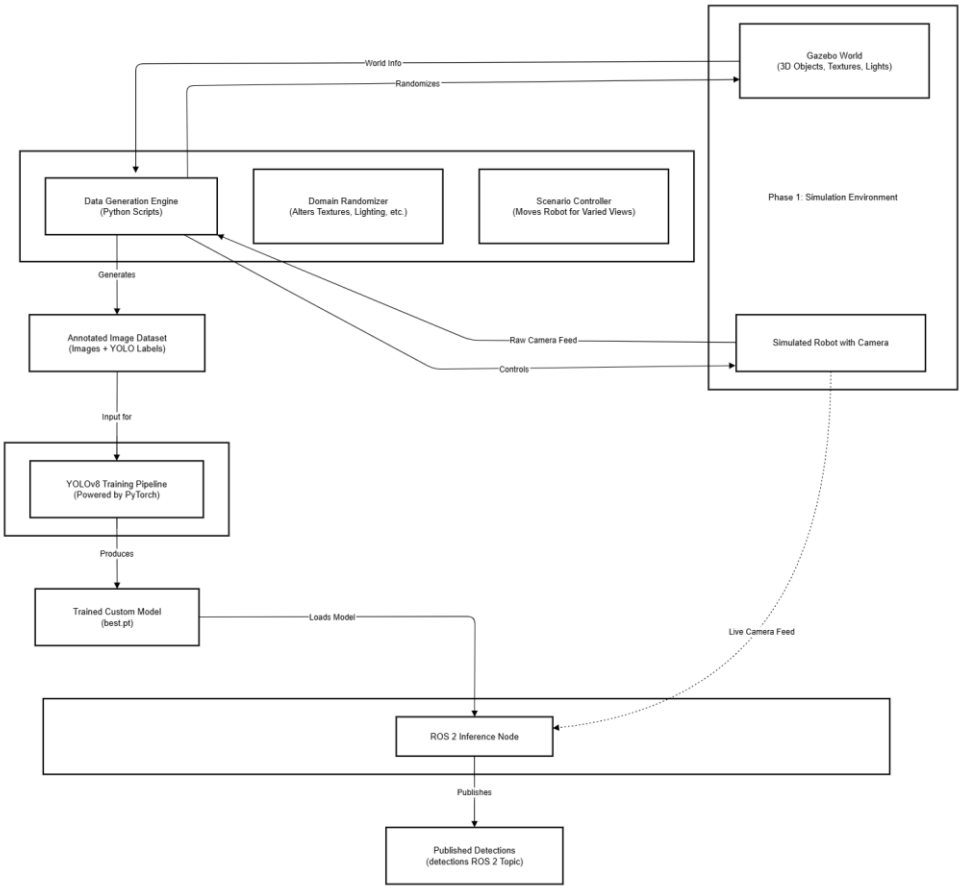

Figure 3-2 Component Diagram

Several common, open-source tools that are extensively utilized in the robotics and artificial intelligence sectors serve as the foundation for this component. This guarantees stability, compatibility, and availability of a large community of support. The planning process starts with a detailed requirements analysis, which is followed by a study to make sure the project is doable given its technical and schedule limitations.

3.2.1 Requirement Gathering and Analysis

The initial phase of the project is dedicated to defining the technical requirements and operational constraints necessary for success. This process ensures the developed perception system aligns perfectly with the needs of the broader autonomous navigation platform.

Primary Data Sources: Requirements are primarily derived from a comprehensive review of academic literature on collaborative SLAM, sim-to-real transfer, and domain randomization techniques. Additionally, the input requirements of the other project components (the Semantic SLAM and Meta-RL modules) are a critical source, defining the specific object classes, data formats, and performance targets the perception system must deliver.

Prototyping and Baselines: Early-stage prototyping using simulated robots in Gazebo is essential. This allows for the establishment of initial performance baselines for key metrics, such as data generation throughput (images per hour) and the inference speed of a pre-trained YOLOv8 model on the target hardware. These metrics will inform the design of the full data generation pipeline.

Analysis of Operational Constraints: A detailed analysis of the project's limitations is performed to ensure the framework's robustness and adaptability:

- **Computational Constraints on Target Hardware:** The object detection model must be capable of running in real-time on the limited onboard processing power of a typical mobile robot. This necessitates choosing an efficient model architecture (YOLOv8) and potentially applying optimization techniques like model pruning or quantization.
- **The Sim-to-Real Gap:** The discrepancy between the virtual and physical worlds is the most significant environmental constraint. The requirement to mitigate this gap mandates the implementation of a robust domain randomization strategy during data generation. The system must be designed to introduce variability in textures, lighting, and object placement.
- **System Integration Interface:** The perception module cannot be a black box. A strict requirement is the definition of a clear and efficient data interface using ROS 2 custom messages. This ensures that the output (detected object classes, confidence scores, and bounding boxes) is seamlessly and reliably consumed by the SLAM and navigation modules.

3.2.2 Feasibility Study

A feasibility study is conducted to ensure that the project's design can satisfy performance, resource, and schedule restrictions, guaranteeing its viability for successful implementation.

Technical Feasibility: This project is considered highly feasible from a technical standpoint. All the core software components (Ubuntu, ROS 2, Gazebo, PyTorch, YOLOv8) are mature, open-source, and well-documented, with proven compatibility. The primary technical challenge training a deep learning model is well within the capabilities of modern consumer-grade GPUs. The process of integrating a trained model into a ROS 2 node is a standard and well-understood task in the robotics community.

Economic Feasibility: The economic feasibility is exceptionally high. By leveraging a fully open-source software stack and generating data synthetically, the project circumvents the two largest expenses in robotics perception research: hardware costs (no need for a physical robot) and manual data annotation services. The only significant cost is the development of computers, which is an existing resource.

Operational Feasibility: The project is operationally feasible. The final output self-contained ROS 2 node is designed for modular integration with the larger team project. The skills required to complete the project (Python programming, Linux command line, basic deep learning concepts) are aligned with the curriculum of a Software Engineering specialization. Extensive documentation and tutorials for all chosen technologies will mitigate the learning curve.

Schedule Feasibility: The methodology's phased approach directly supports schedule feasibility. The project is broken down into logical, sequential milestones (environment creation, data pipeline development, model training, and integration) that can be realistically achieved within the academic timeline. Each phase has a clear deliverable, allowing for consistent progress tracking.

3.3 Tasks and Sub-Tasks

To accomplish Autonomous Navigation Training project goals, the development process is divided into the following tasks and further divided into subtasks to guarantee steady advancement and effective execution.

1. Environment and Simulation Setup
   - Install and configure Ubuntu, ROS 2, and Gazebo.
   - Design and build a simple indoor world file in Gazebo.
   - Configure a TurtleBot3 (or similar) robot model with a camera sensor.
   - Write a ROS 2 launch file to spawn the world and the robot.
2. Data Generation Pipeline Development
   - Write a Python script to control the robot's position in Gazebo.
   - Implement functions to save camera images and automatically generate corresponding YOLO annotation files.
   - Implement domain randomization for lighting and textures.
   - Generate an initial dataset of at least 1,000 annotated images.
3. Model Training and Fine-Tuning
   - Set up the Ultralytics YOLOv8 training environment.
   - Train an initial YOLOv8 model on the generated dataset.
   - Analyze training results and iteratively refine the dataset and training parameters.
4. ROS 2 Integration and Validation
   - Develop a ROS 2 Python node to perform inference using the trained model.
   - Integrate the node to subscribe to the simulated camera and publish detections.
   - Create an RViz2 configuration to visualize the camera feed and the overlaid bounding boxes from the detection topic.
   - Conduct quantitative performance evaluation using a held-out test set.

3.4 Testing and Validation

The success of the project will be measured through a multi-stage validation process:

Unit Testing: Each script and ROS 2 node will be tested individually. For example, the data generation script will be checked to ensure annotations perfectly match the object positions.

Model Performance Metrics: The primary validation will be quantitative. The trained model will be evaluated on a reserved test set of synthetic images. Standard metrics, including Precision, Recall, and mean Average Precision (mAP@.50), will be calculated to assess its accuracy.

Qualitative Validation: The real-time performance will be visually inspected using RViz2. This will confirm that the system correctly identifies objects as the robot moves through the simulated environment and that the bounding boxes are accurate.

Inference Speed Testing: The frames-per-second (FPS) of the ROS 2 inference node will be measured to ensure it meets the real-time requirements for fluid navigation.

3.5 Data Collection

All data used for training and testing the object detection model will be synthetically generated as a core part of this project.

- **Simulated Data:** The Gazebo simulator will be used to generate a dataset of RGB images from a robot-mounted camera in various indoor scenes.
- **Automatic Annotation:** For each image, a corresponding label file will be automatically generated containing the class ID and bounding box coordinates for every target object in the frame.
- **Dataset Structure:** The dataset will be split into training, validation, and test sets (typically an 80/10/10 split) to ensure unbiased evaluation of the model's performance. The initial target is to generate a dataset of 1,000-2,000 images with significant variations introduced through domain randomization.

3.6 Time Frame & Schedule

Figure 3-3: Gantt Chart

4 PROJECT REQUIREMENTS

4.1.1 Functional Requirements

- **Automated Data Generation:** The system must be able to automatically generate a dataset of images and corresponding bounding box annotations from the Gazebo simulation.
- **Custom Model Training:** The system must use the generated dataset to train a custom YOLOv8 model capable of detecting a predefined set of objects.
- **Real-time Object Detection:** The deployed ROS 2 node must be able to process a live camera feed and detect objects in real-time.
- **Publishing of Detection Data:** The node must publish the detection results including class name, confidence score, and bounding box to a ROS 2 topic.
- **Visualization:** The detection results must be visualizable in RViz2, showing bounding boxes overlaid on the camera image.

4.1.2 Non-Functional Requirements

- **Performance:** The inference node should achieve a processing speed of at least 10 frames per second (FPS) on the target hardware to be considered "real-time."
- **Accuracy:** The trained model should achieve a mean Average Precision (mAP@.50) of at least 0.75 on the synthetic test dataset.
- **Modularity:** The data generation scripts, training code, and ROS 2 node should be well-structured and independent, allowing them to be easily modified or reused.
- **Usability:** The entire pipeline should be well-documented, allowing another researcher to replicate the data generation and training process.

4.1.3 System Requirements

- **Development Machine:** A computer running Ubuntu 22.04 with ROS 2 Humble installed.
- **GPU Acceleration:** A CUDA-enabled NVIDIA GPU is required for efficient training of the YOLOv8 model.

- **Software Dependencies:** Python 3.8+, PyTorch, and all libraries specified in the Ultralytics YOLOv8 requirements.

REFERENCES

[1] Yu, C., Liu, Z., Liu, X.-J., Xie, F., Yang, Y., Wei, Q., and Fei, Q., "A Semantic Visual SLAM towards Dynamic Environments," *arXiv preprint arXiv:1809.08379*.

[2] Oriolo, G., Ulivi, G., and Vendittelli, M., "Real-time map building and navigation for autonomous robots in unknown environments," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 28, no. 3, pp. 316-333, 1998.

[3] Xing, Z., Zhu, X., and Dong, D., "DE-SLAM: SLAM for highly dynamic environment," *Journal of Field Robotics*, vol. 39, no. 4, pp. 528-549, 2022.

[4] Ai, Y., Rui, T., Yang, X., He, J., Fu, L., Li, J., and Lu, M., "Visual SLAM in dynamic environments based on object detection," *Journal of Manufacturing Systems*, vol. 60, pp. 257-271, 2021.

[5] Gan F, Xu S, Jiang L, Liu Y, Liu Q, Lan S. Robust visual SLAM algorithm based on target detection and clustering in dynamic scenarios. Front Neurorobot. 2024 Jul 23;18:1431897. doi: 10.3389/fnbot.2024.1431897. PMID: 39108349; PMCID: PMC11300432.

[6] Li, F., Yang, S., Yi, X., and Yang, X., "CORB-SLAM: A Collaborative Visual SLAM System for Multiple Robots," in *Proc. International Conference on Intelligent Robotics and Applications*, 2018, pp. 377-388.

[7] Ahmed, M. F. et al., "Efficient Frontier Management for Collaborative Active SLAM," *arXiv preprint arXiv:2310.01967*, 2023.

[8] Ducatelle, Frederick & Förster, Alexander & Bonani, Michael & Dorigo, Marco & Magnenat, Stéphane & Mondada, Francesco & O'Grady, Rehan & Pinciroli, Carlo & Retornaz, Philippe & Trianni, Vito & Gambardella, Luca Maria. (2014). Cooperative navigation in robotic swarms. Swarm Intelligence. 10.1007/s11721-013-0089-4.

[9] Wu X, Wang G and Shen N (2023) Research on obstacle avoidance optimization and path planning of autonomous vehicles based on attention mechanism combined with multimodal information decision-making thoughts of robots. *Front. Neurorobot.* 17:1269447. doi: 10.3389/fnbot.2023.1269447

[10] Raj, R. and Kos, A., "Intelligent mobile robot navigation in unknown and complex environment using reinforcement learning technique," *Scientific Reports*, vol. 14, no. 1, p. 22852, 2024.

[11] Fu, Z., Kumar, A., Malik, J., and Pathak, D., "Rapidly Adaptable Legged Robots via Evolutionary Meta-Learning," *Google Research*, 2020.

[12] Blanco, Jose Luis. (2019). A Modular Optimization Framework for Localization and Mapping. 10.15607/RSS.2019.XV.043.

[13] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A versatile and accurate monocular SLAM system,” *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

[14] M. Labbé and F. Michaud, "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019. doi: 10.1002/rob.21831.

[15] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2D LIDAR SLAM,” *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 1271–1278, 2016.

[16] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 1126–1135, 2017.

[17] A. Zeng, S. Song, K. Yu, M. K. Wang, and T. Funkhouser, “Learning Synergistic Navigation Policies via Meta-Reinforcement Learning,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2563–2570, Apr. 2020.

[18] T. Huang, M. Kaess, and J. Leonard, “CORB-SLAM: Collaborative Range-Only SLAM for Multi-Robot Systems,” *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 281–288, 2019.

[19] M. Cieslewski, K. S. Liu, and D. Scaramuzza, “Data-Driven Decentralized SLAM for Heterogeneous Robot Teams,” *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2565–2572, Jul. 2018.

[20] N. Sünderhauf, et al., “Place Recognition with ConvNet Features: Static vs Dynamic Environments,” *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2225–2232, Jul. 2018.

[21] D. Kim, J. Park, and S. Lee, “Semantic-Aware Change Detection for Long-Term SLAM,” *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 320–327, 2019.

[22] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2017, pp. 23-30.

[23] J. Tan *et al.*, "Sim-to-Real: Learning Agile Locomotion For Quadruped Robots," in *Proc. Robotics: Science and Systems (RSS)*, 2018.

[24] C. Yu *et al.*, "A Semantic Visual SLAM towards Dynamic Environments," *arXiv preprint arXiv:1809.08379*, 2018.

[25] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

[26] Y. Ai *et al.*, "Visual SLAM in dynamic environments based on object detection," *J. Manuf. Syst.*, vol. 60, pp. 257-271, 2021.

[27] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv preprint arXiv:2004.10934*, 2020.

[28] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1126–1135.